SIMATS ENGINEERING

Assessment Tool 2: Data Interpretation (Medium)

Name : B Yogendra

Reg Number : 192325046

Course Code : CSA4307

Course Name : Internet Programming

Question 1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1.1 Identify any incorrect or invalid fields in the header.

The field Content-Length: -150 is invalid. The HTTP semantics specification (RFC 9110) defines Content-Length as a non-negative decimal integer that gives the exact number of octets (bytes) in the message content. The value is used by the receiver to know where the body ends. A negative number has no meaning in this context, so the field is malformed.

The other fields need to be checked as well. The status line HTTP/1.1 200 OK is correctly formed, with the protocol version, the three-digit status code and the reason phrase. The Date field uses the IMF-fixdate format, which HTTP requires, with the day, month, year, time and the GMT zone. The Content-Type field text/html is a valid media type. The Connection: keep-alive field is permitted in HTTP/1.1, where persistent connections are the default, so it is valid but not necessary. In summary, one field is invalid, and the others are correct.

1.2 Analyze the issue with Content-Length and explain its impact.

Content-Length is the mechanism that lets a client find the end of the body. In a persistent connection, which Connection: keep-alive requests, several responses are sent one after another over the same TCP connection. The client relies on Content-Length to know where one response ends and the next begins. With a negative value, the client cannot determine the body length, so the boundary between responses is lost.

The effect depends on the client, and the test results show this clearly. Figure 1 shows curl reading the same header and stopping with the error 'Invalid Content-Length: value'. The curl client treats the header as a protocol violation and closes the connection. Figure 2 shows Chromium in the same situation. Chromium ignores the invalid length, reads the body and displays the text 'Hello'. This means that the page appears to work, but the browser has made

its own decision about where the body ends, and a different client might reach a different decision.

- **Rejection:** some clients, such as curl, refuse the response, so the user sees an error and no content.
- **Silent tolerance:** some clients, such as Chromium in this test, guess the body length and show the page, which hides the server fault.
- **Waiting:** a client that expects a length may wait for the connection to close, which can delay or freeze the page.
- **Desynchronisation:** on a keep-alive connection, a client that misreads one body boundary may read the next response incorrectly, which is the basis of request smuggling attacks against proxies that disagree about message length.

```
$ curl -sv http://127.0.0.1:8001/
< HTTP/1.1 200 OK
< Date: Mon, 01 Apr 2026 10:00:00 GMT
< Content-Type: text/html
* Invalid Content-Length: value
```

Figure 1: curl rejects the response because Content-Length is negative



Figure 2: Chromium ignores the invalid length and displays the body

1.3 Determine whether the status code matches the response correctness.

The status code 200 OK tells the client that the request succeeded and that the response contains the requested representation. The code itself is valid, and it matches the intention of the server to deliver a page. However, the response as a whole is not correct, because the message that carries the 200 status is malformed. A status code describes the result of the request, while the header describes how the message is to be read. Here the result is success, but the message cannot be read reliably.

The server should not have sent this response. A server that cannot produce a valid header for a successful result must report its own failure. The appropriate status code for this situation is 500 Internal Server Error, which tells the client that the request could not be completed because of a problem on the server. The server's error handling should be checked, since an

invalid length is usually caused by a calculation or a framework bug, and the status code should then reflect that fault.

1.4 Suggest corrections for the identified errors.

The Content-Length field should be replaced by the exact number of bytes in the body. The server should compute the length from the actual body it is about to send, not from an arithmetic expression that can produce a negative value. The corrected header below uses 150 as an example, since the true length depends on the content.

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

There are two further corrections to consider. First, if the body is generated dynamically and its length is not known before it is sent, the server can use Transfer-Encoding: chunked, which sends the body in sized pieces and ends with a zero-length chunk. Second, the error should be logged on the server, and the response should be changed to 500 Internal Server Error if the header cannot be produced correctly.

Question 2: DOM Structure Analysis

The following DOM tree representation is considered:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

2.1 Identify missing or improperly closed elements.

The element that is improperly closed is the element. It is opened before the text 'HelloWorld' but is never closed with . The end tag that follows belongs to the div element, so the paragraph is still open when the div is closed. The markup is therefore nested incorrectly, because the end tag of the outer element arrives while an inner element is still open.

The document has no DOCTYPE declaration, so it has no document type. The html, head, title, body and div elements are opened and closed correctly. The missing DOCTYPE is not a closing-tag error, but it changes how the browser renders the page, as explained in Section 2.2.

2.2 Analyze how the browser will interpret this structure.

HTML parsing is defined as error-tolerant. The browser does not reject the document, because the specification tells parsers how to recover from errors of this kind. When the parser reaches while a is open, it closes the paragraph implicitly, so that the block end tag is not nested inside a paragraph. The parser also creates the implied structure of a document, so that any text outside the head is placed in the body.

Because the document has no DOCTYPE, the browser selects quirks mode. Quirks mode is a compatibility mode that reproduces older browser behaviour, and it changes several layout rules. The parsed DOM confirms both behaviours. The serialized tree below was read from the browser after parsing. It shows the implicit and the document mode reported by the browser.

```
compatMode: BackCompat (quirks mode)
<html><head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</p></div>
</body></html>
```

```
<html>
  <head>
    <title>
      "TestPage"
  <body>
    <div>
      <p>
        "HelloWorld"
```

Figure 3: Parsed DOM tree of the original markup, read from Chromium

2.3 Determine the impact on rendering and layout.

The visible content is not lost. Figure 4 shows the page with the text 'HelloWorld' displayed, so the user sees the content. The effect is on the structure and on how predictable the layout is.

- **Box model:** quirks mode uses the older box model for some elements, so widths and heights of some boxes can differ from the same markup in standards mode.
- **Default styling:** in quirks mode, table cells do not inherit the font size and line height of the body, so text inside tables can look different from the same markup in standards mode.
- **Cross-browser variation:** other browsers recover from the same error with their own rules, so the layout can differ between devices, which makes the page harder to test and maintain.
- **Scripting and styling:** scripts and stylesheets that select elements by their intended structure may fail if the parser has restructured the tree.
- **Validation and accessibility:** a document that fails validation can be misread by assistive technology and search engines, and the missing language attribute also affects speech output.

HelloWorld

Figure 4: The text is displayed, but the markup is invalid and the page is in quirks mode

2.4 Suggest corrections to make the DOM valid.

The corrected version adds a DOCTYPE declaration, which selects standards mode. It closes the paragraph with </p> before the tag. It also adds the language attribute and the character set declaration. The text is kept as 'HelloWorld' so that the comparison is exact.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld</p>
</div>
</body>
</html>
```

The browser now reports standards mode (CSS1Compat), and the DOM tree has the intended shape. Figure 5 shows the tree after correction, in which the paragraph is a child of the div and the meta element appears in the head.

```
<html>
  <head>
    <meta>
    <title>
      "TestPage"
  <body>
    <div>
      <p>
        "HelloWorld"
```

Figure 5: Parsed DOM tree of the corrected markup, read from Chromium

Question 3: GET vs POST in Login System

A login system uses two different methods, as summarised in the table below.

Method	URL example	Data visibility	Security level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

3.1 Interpret the differences in data transmission between GET and POST.

The two methods differ in where the form data is carried in the HTTP request. A GET request has no body. The form fields are encoded and appended to the URL as a query string, after the question mark, so the data is part of the request line. A POST request carries the form fields in the message body, after the blank line that separates the headers from the body. The URL contains only the path of the resource.

The consequences of this difference are visible in the test results. Figure 6 shows the browser after a GET submission. The address field contains the email and password, and the server echoes them back as part of the URL. Figure 7 shows the browser after a POST submission. The URL is clean, and the request body holds the data. Figure 8 shows the server's access log for the two requests, which is the record a server or proxy keeps. The GET entry contains the password in plain text, while the POST entry contains only the path.

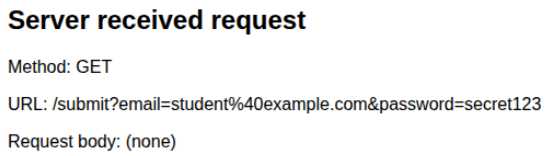


Figure 6: Login submitted with GET. The credentials appear in the URL.

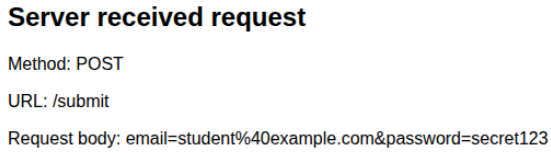


Figure 7: Login submitted with POST. The URL does not contain the credentials.

```
$ tail -n 2 access.log
127.0.0.1 - [12/Oct/2026 15:50:21] "GET /login?user=admin&pass=1234 HTTP/1.1" 200
127.0.0.1 - [12/Oct/2026 15:50:21] "POST /login HTTP/1.1" 200
```

Figure 8: Server access log. The GET request line contains the password; the POST line does not.

3.2 Analyze which method is more suitable for login and why.

POST is the more suitable method for login, for three reasons. First, login sends sensitive values, and the body of a POST request keeps them out of the URL, so they are not stored in browser history, bookmarks, server logs or proxy logs. Second, a login changes server state, because it creates a session. HTTP defines GET as safe and cacheable, meaning it is expected to have no side effects and its response may be stored, so using GET for an action that creates a session is contrary to the method's design. Third, a POST is not cached or replayed from a link, so a login cannot be repeated by someone who copies a URL.

POST is not secure by itself. The body is sent as plain text unless the connection is encrypted with TLS. A network observer can read a POST body just as easily as a GET query string. The correct conclusion is that POST is the appropriate method, and it must be used over HTTPS. The method choice protects the credentials from exposure in URLs and logs, and HTTPS protects them from exposure on the network.

3.3 Identify security risks in using GET for authentication.

- **Exposure in server and proxy logs:** web servers, reverse proxies, load balancers and content delivery networks normally record the full URL of each request. The password is therefore stored in plain text in several places that the developer may not control.
- **Browser history and bookmarks:** a login URL with credentials is saved in the browser history and may be bookmarked, so anyone with access to the device can recover the password.
- **Referer leakage:** when a page loaded from the login URL includes external resources, such as images, scripts or analytics, the full URL can be sent to those third parties in the Referer header.
- **Shoulder surfing and sharing:** the credentials are visible on the screen and can be copied from the address bar or from a link shared with another person.
- **Login CSRF:** because GET requests are triggered by simple navigation and by image or script tags, a third-party site can cause a victim's browser to submit a login with attacker-chosen credentials.
- **Length limits:** URLs have practical length limits, so long passwords or tokens can be truncated, which causes unreliable authentication.

3.4 Suggest best practices for secure login implementation.

- **Use POST for login:** never place credentials, tokens or session identifiers in the URL.
- **Use HTTPS everywhere:** serve the whole site over TLS, redirect HTTP to HTTPS, and enable HTTP Strict Transport Security (HSTS) so that browsers only connect over TLS.

- **Hash passwords:** store passwords only as salted hashes produced by an adaptive algorithm such as bcrypt, scrypt or Argon2. Plain-text or fast unsalted hashes are unacceptable.
- **Secure session cookies:** set the Secure, HttpOnly and SameSite attributes so that the cookie is sent only over TLS, is not readable by scripts, and is not sent on cross-site requests.
- **Protect against CSRF:** include an unpredictable token in each form and verify it on the server, and use the SameSite cookie attribute as a second layer.
- **Limit repeated attempts:** apply rate limiting, account lock-out or increasing delays after failed logins to resist password guessing.
- **Use generic error messages:** return a message such as 'Invalid username or password' so that an attacker cannot tell which field was wrong.
- **Add multi-factor authentication:** require a second factor where practical, so that a stolen password alone is not enough.
- **Keep credentials out of logs:** configure logging so that request bodies and sensitive parameters are not written to log files.

The following header shows how a server should issue a session cookie after a successful login:

```
Set-Cookie: sessionid=8f3a9c; Path=/; Secure; HttpOnly; SameSite=Strict
```

Question 4: HTML Code Inspection

The following HTML snippet is analysed.

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

4.1 Identify missing closing tags.

Two elements are not closed. The <h1> element has no </h1> end tag, and the element has no end tag. The title, head, body and html elements are all closed correctly, and the document has a DOCTYPE. The <img> element is a void element in HTML5, which means it never has content and does not need an end tag, so its absence of a closing tag is correct.

4.2 Analyze issues with the <img> tag.

The img element has a valid src attribute, but it has three weaknesses. The first is the missing alt attribute. Alternative text describes the image for screen-reader users and for anyone who cannot load images, and HTML requires the attribute for meaningful images. The second is

the missing width and height. Without these, the browser does not know the space the image will need, so the text below it moves when the image loads, which is a layout shift that users notice. The third is the source itself. The file image.jpg does not exist in this scenario, so the browser cannot display the image. Figure 9 shows the broken image icon that results.

4.3 Determine how browsers will handle these errors.

Browsers continue to process the document after these errors, and they show no error message to the user. The effect of the unclosed <h1> is the most significant. An h1 element is a heading, so the text inside it is displayed large and bold. Because the h1 is never closed, the paragraph that follows is placed inside the heading. The parser does not close the h1 when it reaches the <p> start tag, so the paragraph becomes a child of the heading and inherits its styling. Figure 10 shows the rendered result, in which 'This is a paragraph' is drawn in heading style. Figure 11 shows the parsed DOM, in which the <p> and the <img> are both inside the <h1>.

The image is inside the paragraph, which is inside the heading. This nesting is not valid, because a heading may contain only phrasing content and a paragraph is flow content. Browsers allow it, but the structure is wrong, and its consequences appear in styling, navigation and accessibility tools. A screen reader that builds a list of headings would announce the paragraph as part of the heading text.

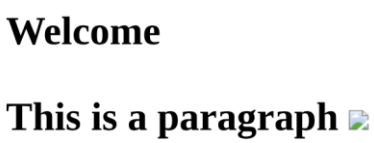


Figure 9: Rendered snippet. The paragraph inherits heading style and the image is missing.

```
<html>
  <head>
    <title>
      "Sample"
  <body>
    <h1>
      "Welcome"
      <p>
        "This is a paragraph"
        <img>
```

Figure 10: Parsed DOM tree of the original snippet. The <p> and <img> are nested inside <h1>.

4.4 Suggest a corrected version of the code.

The corrected version closes both elements, adds the language attribute, the character set and the alternative text, gives the image its dimensions, and ends the paragraph with a full stop. The image source is unchanged, and the file must be placed in the same folder for the image to appear.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph.</p>
<img src="image.jpg" alt="Description of the image" width="300"
height="200">
</body>
</html>
```

```
<html>
  <head>
    <meta>
    <title>
      "Sample"
  <body>
    <h1>
      "Welcome"
    <p>
      "This is a paragraph."
    <img>
```

Figure 11: Parsed DOM tree of the corrected snippet. Each element is a sibling in the intended order.

Question 5: Browser-Server Communication Flow

The following sequence describes a web request.

1. The user enters a URL in the browser.
2. A DNS lookup occurs.
3. An HTTP request is sent.
4. The server processes the request.
5. The response is returned.
6. The browser renders the page.

The sequence is correct in outline, but it omits steps that are necessary for the request to be sent. Before the HTTP request can be sent, the client must open a TCP connection to the server, and for HTTPS it must also complete a TLS handshake. These steps lie between steps 2 and 3. After step 6, the browser requests every script, stylesheet and image that the page refers to, and each of those requests repeats the same communication cycle. The discussion below includes these steps where they affect the answer.

5.1 Interpret each step in the communication flow.

- **Step 1, URL entry:** the browser parses the URL into its scheme (http or https), host name, port and path. The scheme determines whether encryption will be used, and the host name is the value that DNS must resolve.
- **Step 2, DNS lookup:** the browser checks its own cache for the host name. If the name is not cached, the operating system resolver and then a recursive DNS resolver are asked. The result is an IP address, which is needed to reach the server. Figure 12 shows the local resolution step.

- **Step 3, HTTP request:** the client opens a TCP connection to the IP address and port, and for HTTPS negotiates a TLS session. It then sends the request line, with the method and path, followed by the headers, such as Host and Accept. The request is the client's instruction to the server.
- **Step 4, server processing:** the web server selects the code to handle the path, runs any application logic, queries databases or other services, and builds the response. The time taken here is the server's contribution to the total delay.
- **Step 5, response:** the server returns a status line, headers, including Content-Type and Content-Length, and a body. The headers tell the browser how to interpret the body, which is why the header errors in Question 1 matter.
- **Step 6, rendering:** the browser parses the HTML into a DOM, parses the CSS into a CSS object model, combines them into a render tree, calculates layout, and paints the pixels. Scripts can change the DOM during this process.

```
$ getent hosts localhost
127.0.0.1       localhost
---
127.0.0.1       localhost
```

Figure 12: Name resolution of localhost on the local machine

5.2 Identify where delays can occur.

Each stage in the sequence adds latency, and some stages add more than one round trip. The main sources of delay are listed below, with the reason each one matters.

- **DNS lookup:** an uncached name must be resolved through the resolver hierarchy: root, top-level domain and authoritative servers. Each query may take tens or hundreds of milliseconds across the internet.
- **Connection setup:** the TCP handshake takes one round trip before any data is sent, and a TLS handshake adds one or two more round trips. On a distant server, these delays are added before the first byte of the request is sent.
- **Server processing:** slow database queries, unoptimised code and blocking calls to other services increase the time to first byte, which is the time between sending the request and receiving the response.
- **Network transfer:** distance, bandwidth and packet loss determine how long the response takes. A large body takes longer to transfer, and loss causes retransmission.
- **Render-blocking resources:** stylesheets and synchronous scripts in the head must be loaded before the browser can display the page, so a slow external file delays rendering even if the HTML has arrived.
- **Many separate requests:** a page that refers to many scripts, images and fonts makes many requests, and each one adds its own cost, particularly on connections where setup is slow.

The browser timing in Figure 13 shows the stages for a local request. The values are small, because the request does not leave the machine. On a real network, the DNS lookup and

connection set-up are much larger, and the same measurements would show where the delay occurs.

```
Browser navigation timing for http://127.0.0.1:8000/complete.html
(values in milliseconds from the start of navigation)

DNS lookup start (domainLookupStart)          5.00 ms
DNS lookup end (domainLookupEnd)              5.00 ms
TCP connect start (connectStart)              5.00 ms
TCP connect end (connectEnd)                  5.50 ms
Request sent (requestStart)                   5.70 ms
First byte received (responseStart)           6.90 ms
Response complete (responseEnd)               9.90 ms
DOM parsed (domInteractive)                  35.30 ms
Page fully loaded (loadEventEnd)             91.00 ms
```

Figure 13: Navigation timing reported by the browser for one request

5.3 Analyze the role of DNS in the process.

The Domain Name System translates human-readable domain names, such as example.com, into IP addresses, such as 93.184.216.34. Routers use IP addresses to deliver packets, so the client must know the address before it can send the request. DNS therefore sits at the start of the sequence, and every later step depends on it.

The lookup is hierarchical. A recursive resolver asks a root server, which refers it to the server for the top-level domain, which in turn refers it to the authoritative name server for the domain. The authoritative server holds the record and returns the address. Each answer carries a time-to-live (TTL) value, which tells caches how long they may keep the answer. Browsers, operating systems and resolvers cache answers, so most lookups are served without contacting the full hierarchy. Figure 12 shows the local resolution of localhost, which is resolved from the system's hosts file without a network query.

DNS also affects reliability. If the lookup fails, the browser cannot find the server, and the page does not load even when the server is working. Errors in DNS records, expired domains and misconfigured TTLs can therefore make a site unavailable, although the server itself is healthy.

5.4 Suggest optimizations to improve performance.

- **Caching DNS answers:** choose TTL values that balance freshness and speed, so that resolvers can serve repeat lookups from cache.
- **Reuse connections:** use HTTP keep-alive, HTTP/2 or HTTP/3 so that one connection carries several requests, and use TLS session resumption to avoid repeating the full handshake.
- **Reduce round trips:** combine files where practical, and use preconnect or dns-prefetch hints for third-party domains the page will need.
- **Compress responses:** compress text responses such as HTML, CSS and JavaScript with gzip or Brotli.
- **Cache responses:** send Cache-Control headers and ETag or Last-Modified validators so that browsers can reuse unchanged resources.
- **Speed up the server:** optimise database queries, add indexes, and cache results that change rarely, so the time to first byte is reduced.

- **Use a content delivery network:** serve static files from servers near the user, which reduces both transfer time and round-trip time.
- **Improve rendering:** place stylesheets early, load scripts with defer or async, minify and bundle assets, and lazy-load images below the fold.